Graphic Era HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)

DEHRADUN CAMPUS

PROJECT REPORT

REAL TIME FACE RECOGNITION SYSTEM

B.Tech (CSE)

4th Semester

SUBMITTED BY:

NAME: KANISHK KUMAR

UNIVERSITY ROLL No. - 2218917

CLASS ROLL No. - 35

SECTION: K1

COURSE: B.Tech (CSE)

SUBMITTED TO:

Dr. Mohit Payal

Dept. of Computer science & Engineering

Graphic Era Hill University

Graphic Era HILL UNIVERSITY

Established by an Act of the State Legislature of Uttarakhand (Adhiniyam Sankhya 12 of 2011)

DEHRADUN CAMPUS

CERTIFICATE

University Roll No.: 2218917 Class Roll No.: 35

Certified that KANISHK KUMAR has developed mini project on "Real Time Face Recognition System" for the CS 4th semester Mini Project Lab in Graphic Era Hill University, Dehradun. The project carried out by Students is their own work as best of my knowledge.

DATE- 20/07/2024

Dr. Mohit Payal

Class Co-ordinator

CSE-K1-IV-Sem

(CSE Department)

GEHU Dehradun

ACKNOWLEDGMENT

I would like to express our gratitude to The Almighty Shiva Baba, the most Beneficent and the most Merciful, for completion of project. I wish to thank my parents for their continuing support and encouragement. I also wish to thank them for providing us with the opportunity to reach this far in our studies. I would like to thank particularly our project Co- Dr. Mohit Payal for her patience, support and encouragement throughout the completion of this project and having faith in me.

At last but not the least I greatly indebted to all other persons who directly or indirectly helped us during this work.

KANISHK KUMAR

Roll No.- 2218917

B.TECH (CSE)

K1-IV-Sem

Session- 2022-2026

GEHU, Dehradun

TABLE OF CONTENTS

- **Executive Summary**
  - Overview of the face recognition system and its core functionalities.

- **Introduction**
  - Background and relevance of face recognition technology.
  - Purpose and significance of the project.

- **Objectives**
  - Primary goals of the project.
  - Specific tasks and achievements aimed for in the system development.

- **System Architecture**
  - High-level structure of the system.
  - Description of major components and their interactions.

- **Implementation Details**
  - Detailed explanation of libraries and tools used.
  - Step-by-step breakdown of the code and its functionalities.

- **System Workflow**
  - Comprehensive workflow of the system from initialization to continuous operation.
  - Detailed steps involved in processing and recognizing faces.

- **Future Enhancements**
  - Potential improvements and additions to the system.
  - Long-term vision for the development and application of the system.

- **Conclusion**
  - Summary of the project outcomes.
  - Reflection on the effectiveness and potential of the face recognition system.

- **References**
  - List of sources and references used in the project.
  - Links to libraries, tools, and documentation.

Executive Summary

This report provides an in-depth analysis of a real-time face recognition system that combines OpenCV for video processing, face_recognition library for facial identification, and Tkinter for creating a graphical user interface (GUI). The system captures live video feeds from a webcam, processes the video to detect and recognize faces, and displays the results in an interactive and user-friendly interface. The primary aim is to demonstrate the application of computer vision and machine learning techniques in a practical, real-world scenario. The system showcases efficient face detection and recognition, making it suitable for various applications such as security systems, user authentication, and automated attendance tracking.

Introduction

Face recognition technology has become increasingly prevalent in various domains, including security, surveillance, and personal device authentication. The ability to identify individuals in real-time provides significant advantages in ensuring security and enhancing user experience. This project leverages the power of OpenCV, face_recognition library, and Tkinter to create a real-time face recognition system. The system captures video from a webcam, detects faces in the video stream, matches them against a preloaded database of known faces, and displays the results in an intuitive GUI. The project aims to highlight the seamless integration of these technologies to achieve efficient and accurate face recognition.

Objectives

The primary objectives of this project are as follows:

1. **Develop a Real-Time Face Recognition System**: Create a system capable of recognizing faces in real-time using live video feeds from a webcam.
2. **Integrate OpenCV for Video Processing**: Utilize OpenCV to capture, process, and display video frames.
3. **Use face_recognition Library for Face Identification**: Employ the face_recognition library to detect and recognize faces by comparing them with known face encodings.
4. **Create a User-Friendly GUI with Tkinter**: Design a graphical user interface using Tkinter to display the video feed and recognized names, making the system accessible and easy to use.
5. **Provide Real-Time Updates**: Ensure the system continuously updates the GUI with the latest recognition results, reflecting the dynamic nature of the video feed.

System Architecture

The system architecture is designed to efficiently handle multiple tasks simultaneously, ensuring real-time face recognition and responsive user interaction. The architecture consists of the following components:

1. **Face Encoding Loader**: This module loads images from a specified directory, extracts face encodings, and stores them for later comparison. The face encodings are generated using the face_recognition library.
2. **Video Capture Module**: This component uses OpenCV to capture live video feed from the webcam. It continuously retrieves frames from the webcam for processing.
3. **Face Recognition Module**: This module processes each video frame to detect faces and compare them with known face encodings. It uses the face_recognition library to perform face detection and recognition.
4. **Graphical User Interface (GUI)**: The GUI is created using Tkinter and displays the video feed along with the names of recognized individuals. It includes various widgets to present real-time information in an organized manner.

Implementation Details

Libraries and Tools:

- **OpenCV**: An open-source computer vision library used for real-time video capture and processing. It provides tools for image and video analysis, making it ideal for this project.
- **face_recognition**: A Python library built on top of dlib, designed specifically for face recognition tasks. It offers high-level functions to detect, encode, and compare faces.
- **Tkinter**: Python's standard GUI toolkit used to create the graphical user interface. It allows for the development of interactive applications with ease.
- **PIL (Python Imaging Library)**: A library for opening, manipulating, and converting images. It is used to convert OpenCV images to a format compatible with Tkinter.

Code Breakdown:

1. **Importing Libraries**:
    - The script begins by importing necessary libraries for video processing, face recognition, and GUI creation.
2. **Function Definition**:
    - get_face_encoding(image_path): This function loads an image from a given path, processes it using face_recognition to extract face encodings, and returns the encoding.
3. **Loading Known Face Encodings**:

- The script iterates through the persons directory, loads images, extracts face encodings, and stores them along with corresponding person IDs.

4. **Initializing Webcam**:
    - OpenCV is used to initialize the webcam and capture video frames.
5. **Creating the GUI Window**:
    - A Tkinter window is created and configured, with labels and list boxes to display the video feed and recognition results.
6. **Updating the Video Feed**:
    - The update_video function captures and processes video frames, detects and recognizes faces, and updates the GUI components accordingly.
7. **Running the Tkinter Main Loop**:
    - The Tkinter event loop is started to keep the GUI responsive and continuously update with new recognition results.

System Workflow

The system workflow involves several steps to ensure efficient face recognition and user interaction:

1. **Initialization**:
    - Load known face encodings from the persons directory.
    - Initialize the webcam for real-time video capture.
    - Set up the Tkinter GUI components.
2. **Video Processing**:
    - Capture video frames from the webcam using OpenCV.
    - Detect face locations and extract face encodings from each frame.
    - Compare detected face encodings with known encodings to identify recognized faces.
3. **Display Results**:
    - Draw rectangles around recognized faces and display labels with their names.
    - Update the GUI with the names of recognized individuals in the current frame.
    - Maintain a list of all recognized names and update it dynamically.
4. **Continuous Operation**:
    - The update_video function is called repeatedly to process and display new frames in real-time.
    - The Tkinter main loop ensures the GUI remains responsive and updates continuously.

Future Enhancements

1. **Improved Recognition Algorithms**:
   - Integrate more advanced machine learning models and techniques to enhance face recognition accuracy and performance.
2. **Multiple Camera Support**:
   - Extend the system to handle multiple video feeds simultaneously, allowing for broader coverage and application.
3. **Enhanced User Interface**:
   - Add more interactive features and customization options in the GUI to improve user experience.
4. **Database Integration**:
   - Store recognized faces and corresponding timestamps in a database for future reference, enabling features like attendance tracking and historical data analysis.
5. **Mobile Application**:
   - Develop a mobile version of the application to provide on-the-go face recognition capabilities, increasing the system's accessibility and usability.

Conclusion

The real-time face recognition system developed in this project successfully demonstrates the integration of computer vision and machine learning techniques to achieve efficient and accurate face recognition. By leveraging OpenCV, face_recognition library, and Tkinter, the system provides a practical solution for real-time face recognition tasks, making it suitable for applications such as security systems, user authentication, and automated attendance tracking. Despite challenges related to lighting conditions and computational demands, the system performs effectively and offers potential for future enhancements. This project serves as a foundation for further exploration and development in the field of face recognition technology.

References

- OpenCV - https://opencv.org/
- face_recognition - https://github.com/ageitgey/face_recognition
- Tkinter - https://docs.python.org/3/library/tkinter.html
- PIL (Python Imaging Library) - https://pillow.readthedocs.io/en/stable/